

INTERNAL BUILD REPORT

MTSAi Traffic Data Lake

What was built, how it works, and what's been verified in the Test environment

Reference document	TDR-004 / DL-PLAN-001, "MTSAi Traffic Data Lake" brief v0.1
Scope of this report	Phase 0 & Phase 1 implementation, Test account only
Target AWS account	Test — 690293068614 (ap-south-1)
Repository	mtsai-datalake
Report date	14 September 2026

Phase 1 — Applied & Verified (Test)

Phase 0 — Blocked on DB access

Dev — Out of scope

§ Contents

1.	Summary
2.	Target Architecture
3.	Data Zones & Storage Layout
4.	Consumer Isolation Model
5.	CI/CD & Deployment Pipeline
6.	What Has Actually Been Built
7.	Verification Results
8.	Bugs Found & Fixed During Verification
9.	Outstanding Items

1 Summary

MTSAi's traffic data (camera events, trip events, third-party feeds) lives in a single Postgres database (RDS) that also runs the live production application. Analytical queries over that data compete with production traffic, are expensive to keep in RDS long-term, and several teams (analytics, forecasting, dashboards, auditors) all need access without touching the primary database directly. This project builds a minimal, serverless data lake on AWS S3 to solve that — storing cold data in an open format, queryable via SQL, with per-team access isolation and a path to per-record deletion for compliance (DPDPA/GDPR/PIPEDA).

This report covers what has actually been **built and verified** in the Test AWS account, not just what's planned. Everything below reflects real infrastructure that exists today and has been tested with live AWS API calls — not a design document.

2 Target Architecture

All components are AWS-managed and serverless except the nightly export job, which runs as a scheduled container. Data flows one direction: from the operational database, into cold storage, out through a SQL query layer to each consumer.

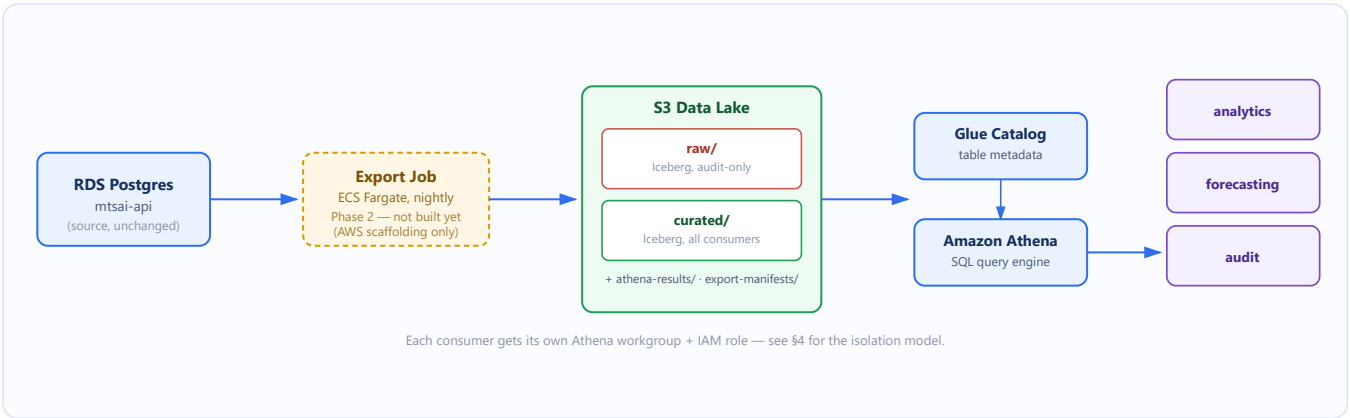


Figure 1 — end-to-end data flow, source to consumer. Dashed box = not yet built (Phase 2).

3 Data Zones & Storage Layout

The lake uses two zones only, per bucket `mtsai-datalake-test-690293068614-ap-south-1` :

Zone	Purpose	Who can read it	Format
<code>raw/</code>	Faithful, unmodified copy of source rows. Audit/replay layer.	audit workgroup only	Apache Iceberg
<code>curated/</code>	Cleaned, deduplicated, analytics-ready tables.	All consumers (analytics, forecasting, audit)	Apache Iceberg
<code>athena-results/<consumer>/</code>	Per-consumer query output, 7-day expiry.	Only that consumer	CSV/metadata
<code>export-manifests/</code>	Reconciliation & audit trail for each export run.	Export job only (Phase 2)	JSON

Apache Iceberg (not plain Parquet folders) was chosen specifically because it supports row-level deletes — required for per-person erasure requests — plus schema evolution and time travel, without locking the lake into one query engine.

4 Consumer Isolation Model

Every consumer gets its own Athena workgroup *and* its own IAM role — never a shared identity. Access is enforced by IAM policy on the underlying S3 prefixes and Glue catalog resources, not by which workgroup happens to be selected in a console.

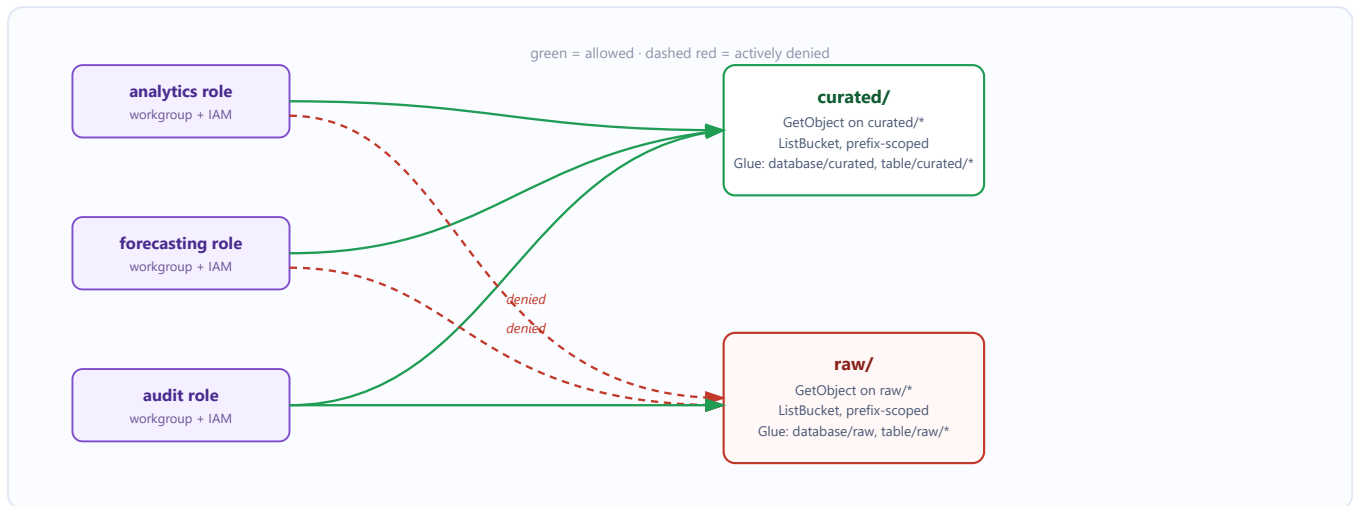


Figure 2 — verified access matrix: only **audit** can reach **raw/** ; all three reach **curated/** .

Why this needed three separate IAM layers, not one

Isolation had to be correct at every layer a query actually touches — S3 storage, the Glue metadata catalog, and Athena's own bucket-verification step — because a gap in any single layer breaks the whole guarantee. This was discovered empirically, not designed up front (see §8).

5 CI/CD & Deployment Pipeline

Infrastructure is provisioned via Terraform, applied through GitHub Actions using short-lived OIDC credentials — no long-lived AWS keys stored anywhere in CI.

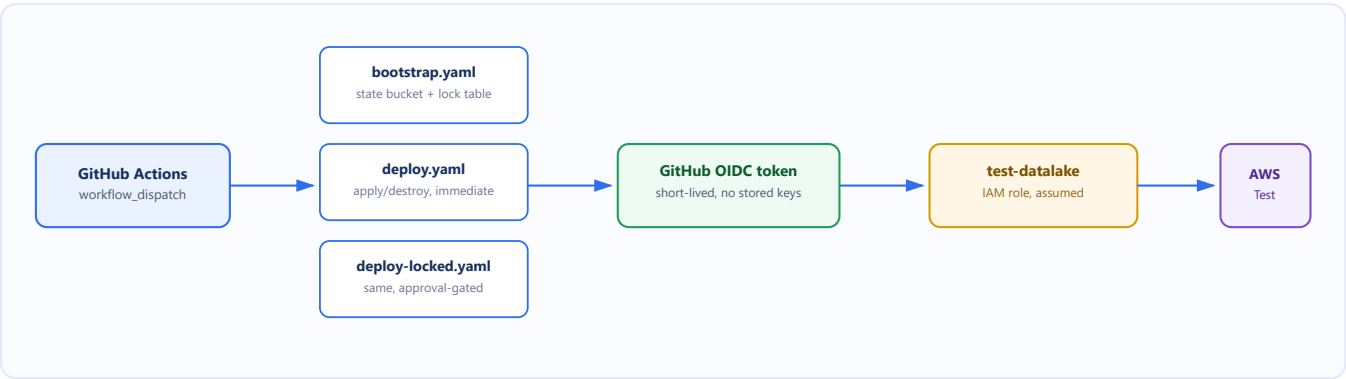


Figure 3 — deployment flow. Both deploy workflows share identical steps; only the GitHub Environment gate differs.

Workflow	Trigger	Approval gate	Purpose
bootstrap.yaml	Manual	None	Creates the Terraform state S3 bucket + DynamoDB lock table, once per account
deploy.yaml	Manual	None	Runs terraform apply or destroy immediately
deploy-locked.yaml	Manual	GitHub Environment reviewer	Same as above, but pauses for approval — used for anything destructive
debug-oidc.yaml	Manual	None	Decodes the OIDC token's claims to debug an IAM trust policy, without calling AWS

6 What Has Actually Been Built

Phase	Status	Notes
Phase 0 — Discovery	Blocked	No Postgres (mtsai-api) DB access from this environment yet; templates ready to fill in once access exists.
Phase 1 — Foundation	Applied & verified (Test)	S3 bucket, KMS key, Glue catalog, 3 Athena workgroups + IAM roles, ECS scaffolding, alarms — all live in Test.
Phase 2 — Export pipeline	Not started	Go export job doesn't exist yet; ECS task definition uses a placeholder image.
Phase 3 — Governance & erasure	Not started	Erasure runbook drafted, not yet rehearsed.
Phase 4 — Postgres trim	Not started	Depends on Phase 2/3.

Live in Test (690293068614)

S3 lake bucket · KMS key · Glue `test_raw` / `test_curated` databases · 3 Athena workgroups (analytics, forecasting, audit) with scoped IAM roles · ECS cluster + task definition (placeholder image) · SNS/CloudWatch failure alerting

Deliberately not built yet

Dev environment (out of scope per direction) · real export container image · VPC subnets for the nightly schedule · Postgres secret · production promotion

7 Verification Results

Isolation was verified by actually assuming each consumer's IAM role via STS and running real Athena queries — not by switching workgroups while signed in as an admin, which doesn't test anything (the workgroup selection doesn't change which identity authorizes the query).

Role	Query against <code>curated</code>	Query against <code>raw</code>	Expected raw access?	Result
<code>analytics</code>	SUCCEEDED	FAILED (AccessDenied)	No	Correct
<code>forecasting</code>	SUCCEEDED	FAILED (AccessDenied)	No	Correct
<code>audit</code>	SUCCEEDED	SUCCEEDED	Yes	Correct

Repeatable via `runbooks/isolation-check.ps1`, committed to the repo — assumes all three roles automatically and reports pass/fail against expectations.

8 Bugs Found & Fixed During Verification

- None of these were caught by `terraform plan` or code review — all four were only findable by actually exercising the deployed permissions with real API calls.
- S3 key enumeration leak.** `s3:ListBucket` was granted on the bare bucket ARN alongside prefix-scoped `s3:GetObject`. Since `ListBucket` is a bucket-level action, this let every consumer list (though not read) keys under `raw/` regardless of their actual read permissions. Fixed by scoping `ListBucket` with an `s3:prefix` condition per zone.
 - Missing `s3:GetBucketLocation`.** Athena calls this on the results bucket before running *any* query, for every consumer. It was never granted anywhere, so every query failed immediately with "Unable to verify/create output bucket."
 - Tag-based Glue permissions don't work for hand-created tables.** Table reads were conditioned on an `Environment` resource tag, but tables are created by hand via Athena DDL/CTAS, not by Terraform — they never receive that tag. This silently denied `glue:GetTable`, which Athena confusingly surfaced as `TABLE_NOT_FOUND` rather than an access error. Fixed by scoping Glue permissions with explicit ARNs (`database/<name>` , `table/<name>/*`) instead of tags — this also properly isolates raw-zone *metadata* by zone, which the tag approach never did.
 - Glue's hierarchical authorization.** Reading a table requires an explicit Allow at every level of catalog → database → table, not just the most specific ARN. `GetDatabase` was granted at the catalog level but `GetTable` / `GetTables` / `GetPartition(s)` were not — blocking legitimate curated-zone reads even after every other permission was correct.

All four fixes are applied and re-verified in Test as of this report (see §7).

9 Outstanding Items

Item	Why it's not done
Nightly export schedule	Needs real VPC subnet/security-group IDs for Test — not yet confirmed. Schedule is deliberately skipped (not failing) until then.
Postgres read access	No connection details or DB client available yet — blocks Phase 0 discovery entirely.
Real export container image	Phase 2 (Go export job) hasn't been written yet.
AWS Budgets / cost alerts	Needs an alarm email and confirmation that cost-allocation tags are active org-wide.
Plan-on-PR / apply-on-merge CI	Current workflows are manual-dispatch only; automatic PR-triggered plans aren't built yet.